

SEPARATING OPTIMIZER-STATE INTERVENTIONS FROM UPDATE-SCALE EFFECTS IN CONTINUAL LEARNING

Anonymous authors

Paper under double-blind review

Internal working draft v0 — not submitted, not under review. The running header is printed by the unmodified official ICLR 2027 style in anonymous mode. All numbers are generated from raw result files by `paper/tools/export_numbers.py`; claim-to-evidence links are in `paper/claim_evidence.tsv`. Human review: pending.

ABSTRACT

Resetting or rescaling Adam’s moment estimates or its bias-correction counter has been proposed as an inexpensive remedy for the loss of learning ability under non-stationarity. Such optimizer-state interventions also change the size of subsequent updates, so their effect is confounded with update scale. We examine this confound in a small, fully paired synthetic setting. First, we observe the phenomenon: on two synthetic 50-task streams and three seeds, a network that has trained on the whole stream learns a fixed, never-seen probe task worse than the same network after its first task, with identical probe batches, dropout masks and update budget (difference in probe learning-curve area -0.245 and -0.204). Second, from that late checkpoint we branch state-only interventions with exact per-moment bias correction and compare each to a control that keeps the original optimizer state but copies the intervention’s per-step, per-tensor update norms. No correctly bias-corrected intervention exceeded its update-norm control by our pre-declared minimum interesting effect of 0.02. Replacing the optimizer state by a fresh one did not recover the gap; resetting only the counter is, with $\epsilon = 0$, a per-tensor learning-rate schedule; and resetting only the second moment produced very large updates despite exact bias correction. The largest gains came from a deliberately mis-corrected reset with several-fold larger updates, for which no learning-rate-matched control was run. These are observations under oracle intervention timing, one probe task per condition, three previously inspected seeds and no real data; they do not show that optimizer-state repair cannot help.

1 INTRODUCTION

Networks trained on long sequences of tasks can lose the ability to learn new ones (Dohare et al., 2024; Lyle et al., 2023). Adam (Kingma & Ba, 2015) is a recurring suspect: its second-moment estimate adapts slowly after an abrupt change, producing unusually large or unusually small updates (Lyle et al., 2023; Dohare et al., 2024; Ellis et al., 2024). This has motivated interventions that act only on the optimizer state, leaving the weights untouched: resetting both moments and the counter at known boundaries (Asadi et al., 2023), resetting only the step counter (Ellis et al., 2024), or carrying and mixing second moments across tasks (Kang & Lee, 2024).

An intervention on Adam’s state changes the magnitude of the next updates as well as their direction. If an intervention helps, the benefit may therefore come from “repairing” stale statistics or simply from taking larger (or smaller) steps. Ellis et al. (2024) already note that their counter reset behaves like learning-rate annealing or warm-up. We ask a narrower diagnostic question: *in a setting where a loss of learning ability is actually observed on a comparable new task, does a correctly bias-corrected optimizer-state intervention help beyond what matching its update scale explains?*

We answer it in a deliberately small, fully instrumented setting. All branches start from one checkpoint, see the same batches and dropout masks, and are checked to differ from the untouched checkpoint only in optimizer state at the moment of intervention. Each intervention is paired with an update-norm control that uses its own optimizer state for the direction and the intervention’s per-step, per-tensor norms for the magnitude. Our findings are:

- **The phenomenon is present in our setting.** On two synthetic 50-task streams, the late network’s probe learning-curve area is lower than the early network’s by -0.245 (random teacher) and -0.204 (label permutation), with every seed negative (Section 5.1).
- **We found no support for a benefit of correctly corrected state-only interventions beyond update scale.** A fresh optimizer state on the late weights did not recover the gap. The only intervention with a consistent positive difference to `keep_all`, resetting the first moment, exceeded its update-norm control by less than our minimum interesting effect (Section 5.2). This is absence of support at three seeds, not evidence of zero effect.
- **The algebra of counter and moment policies explains two of the observations** (Section 4). With $\epsilon = 0$ a counter-only reset rescales every coordinate of a tensor by the same factor, so it is a learning-rate schedule. With $\epsilon > 0$ this equivalence is only approximate. Exact bias correction of a reset second moment does not bound the update, because the kept first moment can be large where the current gradient is small. These are elementary consequences of the Adam update, stated for clarity rather than as new results.

We do not propose a new optimizer, do not claim to restore plasticity, and do not claim that features stay fixed. The weights of every branch are updated after the intervention.

2 RELATED WORK

Loss of plasticity. Dohare et al. (2024) document a loss of learning ability across many tasks. They report that Adam with default β values worsens it relative to SGD, and they track dead units, weight magnitude and effective rank. Lyle et al. (2023) show that such diagnostics change the sign of their correlation with plasticity across settings, and they relate plasticity loss to curvature. Lyle et al. (2024) separate pre-activation shift from parameter-norm growth. Wang et al. (2026) predict trainability from gradient statistics rather than rank measures. Their probe discards the Adam state, and no optimizer statistic is among their predictors. Remedies that act on the weights include regenerative regularisation (Kumar et al., 2024), soft parameter resets (Galashov et al., 2024) and paired fast/slow networks (Lee et al., 2024). Klein et al. (2024) survey the reinforcement-learning literature.

Optimizer state under non-stationarity. Lyle et al. (2023) attribute large post-shift updates to $\hat{m}$ adapting faster than $\hat{v}$, and mitigate them with a larger ϵ and a smaller β_2 . In a toy reinforcement-learning setting they report that resetting only the optimizer state did not improve plasticity. Dohare et al. (2024) use $\beta_1 = \beta_2$ in reinforcement learning. Lyle et al. (2024) show one example in which resetting Adam before a new task reduces dead units. Asadi et al. (2023) reset m , v and the counter at every target-network update and find the second moment responsible for most of the gain. Ellis et al. (2024) reset only the counter and derive the post-shift update magnitude with and without this reset. They report that a full reset was worse than plain Adam in their DQN experiments, and they relate their method to learning-rate schedules. Kang & Lee (2024) study resetting, carrying and averaging m and v separately, with forgetting as the focus. Hernandez-Garcia et al. (2026) reset the optimizer and restart warm-up as a *control* to separate weight-level plasticity loss from stale state. Sahu et al. (2026) bound the tracking error of Adam and SGD under drift.

Position of this draft. Everything above is prior work: the observation that optimizer state matters, the individual reset variants, and the algebra of update magnitude after a gradient shift. Within the sources we checked, we did not find a paired comparison of these variants against per-step update-norm controls on a probe task with an observed loss of learning ability. That comparison is the only thing this draft adds. It is a small diagnostic, not a new method.

3 PROBLEM AND EXPERIMENTAL SETTING

Streams. Inputs are $x \sim \mathcal{N}(0, I_{16})$. We use two conditions, both fixed before running. In *random-teacher* ($C=4$ classes), task k labels x by $\arg \max_c (W_k x)_c$ with an independent Gaussian teacher W_k . In *label-permutation* ($C=10$), a single teacher is shared and task k applies a random label permutation. We use 10 classes because with 4 classes only 24 permutations exist, so a 50-task stream would repeat them and the probe task would not be new. A runtime check verifies that the probe permutation never occurs in training. Each stream has 50 training tasks of 200 steps with batch size 16, and one additional task that is never trained in the stream and serves as the probe.

Model and optimizers. The model is a one-hidden-layer MLP ($16-32-C$) with BatchNorm (momentum 0.1), ReLU and dropout 0.1, trained with mean cross-entropy and no replay, regulariser or task identity. The base optimizer is Adam ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$, no weight decay, no AMSGrad). Its learning rate is selected on two development seeds by online training accuracy over the whole stream, from $\{10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}\}$. A single one-sided extension was allowed if the choice fell on a grid edge. The selected values were 0.003 (random teacher) and 0.003 (label permutation), both interior grid points. Adam with $\beta_1=\beta_2=0.9$ and SGD with momentum 0.9 were tuned in the same way and are reported as secondary families (Appendix D).

Probe protocol. From each starting point we train for 200 steps on the probe task and evaluate held-out probe accuracy every 10 steps. The three starting points are: *fresh*, the initial weights of the stream run with a fresh optimizer; *early*, the end of task 0 with its optimizer state; and *late*, the end of task 49 with its optimizer state. All starting points replace their data and dropout random streams with the same probe streams, so they see identical batches and masks (checked by hashes). Because the probe task is the same for all starting points, task difficulty cancels in the late-versus-early comparison. The primary metric is *probe AUC*, the mean held-out probe accuracy over the 20 evaluation points. We also report the first-50-step mean, final accuracy, and the accuracy drop on the starting point’s last training task.

Interventions and controls. Each branch restores the late checkpoint and applies one intervention to the optimizer state. At the moment of intervention we verify bitwise that weights, Batch-Norm buffers, random-number states, stream position, hyperparameters and train- and eval-mode outputs equal those of the untouched checkpoint. The interventions are: `keep_all`; `reset_m` ($m \leftarrow 0$); `reset_v` ($v \leftarrow 0$); `reset_both` (both moments); `reset_t` (counter only, as in Ellis et al., 2024); and two *mis-corrected* artifact controls, `reset_v` and `reset_both` with the counter left unchanged (“shared-keep”). The first three resets use per-moment counters (Section 4). For every primary intervention X , the *update-norm control* `keepdir_Xmag` keeps the untouched optimizer state and computes its own update. At each step it rescales that update, tensor by tensor, to the norm that X ’s branch applied at the same step. The control therefore needs a completed run of X first (one extra 200-step probe) and reads privileged per-step norms. It is a diagnostic, not a deployable method. All interventions are applied at a pre-specified oracle boundary, the end of the stream. This is not a boundary-free or online method.

Decision rules (fixed before running). The minimum interesting effect is 0.02 probe AUC. The phenomenon is *established* for a condition if the mean late-minus-early AUC is at most -0.02 and every seed is negative. An intervention X has an *effect* if its mean difference to `keep_all` is at least 0.02 and every seed is positive. A *difference remains* if, in addition, $|X - \text{keepdir_Xmag}|$ is at least 0.02 on average with a consistent sign across seeds. We use three test seeds (3000–3002). These seeds were already inspected in an earlier smoke run, so this is an exploratory study, not a confirmatory one. The run used 2,078 s of CPU under a 3,600 s cap and passed all 72 planned cells (code 0404c14).

4 ANALYSIS: MOMENT AND COUNTER POLICIES

We write out the Adam update in a form that separates moment values from counters. Everything in this section follows directly from the update rule of Kingma & Ba (2015). We state it to fix precisely what the interventions and controls do. It is not a new result.

Update with per-moment counters. Let $a_s = 1 - \beta_1^s$ and $b_s = 1 - \beta_2^s$. After the moment update with gradient g , let m, v be the (post-gradient) moments. Standard Adam uses one counter s for both moments. We allow separate counters s_m, s_v , which coincide in ordinary training. The update of coordinate i is

$$u_i = -\alpha \frac{m_i/a_{s_m}}{\sqrt{v_i/b_{s_v}} + \epsilon} = -\alpha \frac{\sqrt{b_{s_v}}}{a_{s_m}} \cdot \frac{m_i}{\sqrt{v_i} + \epsilon \sqrt{b_{s_v}}}. \quad (1)$$

This assumes ϵ is added to $\sqrt{v}$, as in Algorithm 1 of Kingma & Ba (2015) and in `torch.optim.Adam`, and not the rescaled $\hat{\epsilon}$ variant. It also assumes no weight decay (L2 or decoupled), no AMSGrad, fixed β_1, β_2, α , and exact arithmetic. Our implementation matches `torch.optim.Adam` under these options (Appendix A). Under a *decoupled* policy, a reset moment restarts its own counter at 0, so $1 - \beta^s$ equals the EMA weight that moment actually carries. Under *shared-keep*, the counter is not changed. This is what zeroing `exp_avg_sq` alone does in PyTorch.

Counter changes with equal moments. Consider two optimizer states with the same post-gradient moments m, v and counters t and r (both moments sharing each counter). For a coordinate with $v_i > 0$, equation 1 gives

$$\frac{u_i^{(r)}}{u_i^{(t)}} = \kappa(r, t) \cdot \frac{\sqrt{v_i} + \epsilon \sqrt{b_t}}{\sqrt{v_i} + \epsilon \sqrt{b_r}}, \quad \kappa(r, t) = \frac{\sqrt{b_r} a_t}{a_r \sqrt{b_t}}. \quad (2)$$

If $\epsilon = 0$, the ratio equals $\kappa(r, t)$ for every coordinate with $v_i > 0$. Coordinates with $v_i = 0$ have received only zero gradients, so $m_i = 0$ and both updates vanish. Hence, when $\epsilon = 0$, changing the counter while keeping the moments multiplies the whole update by one scalar. Our counters are shared by all tensors, so the scalar is also shared by all tensors. For $r = 1$ and $t \rightarrow \infty$, $\kappa \rightarrow \sqrt{1 - \beta_2}/(1 - \beta_1) = 0.316228$ for $(0.9, 0.999)$. For finite t the extra factor $a_t/\sqrt{b_t}$ applies; for example, $\kappa(1, 10,001) = 0.316235$. The limit value is therefore an approximation, not an identity. If $\epsilon > 0$, the second factor of equation 2 depends on v_i . It moves from $\kappa(r, t)$ when $\sqrt{v_i} \gg \epsilon$ to a_t/a_r when $\sqrt{v_i} \ll \epsilon \sqrt{b_r}$. The scalar equivalence therefore breaks for coordinates whose gradient scale is comparable to ϵ (Appendix B).

Consequences for three arms. (i) *reset_t*. The counter reset leaves the moment recursions unchanged. If $\epsilon = 0$, induction over steps (equal weights $\Rightarrow$ equal gradients $\Rightarrow$ equal moments) shows that `reset_t` is `keep_all` run with the deterministic learning-rate multiplier $\kappa(k, t+k)$ at the k -th step after the reset. For $t = 10,000$ this multiplier falls to 0.152 at $k = 12$ and is 0.426 at $k = 200$. A per-tensor update-norm control that uses `keep_all`'s own direction then reproduces `reset_t` exactly in exact arithmetic. For this arm the control is the same algorithm, not an independent test. (ii) *Shared-keep artifacts*. After `reset_both`, the shared-keep and decoupled branches start from the same zero moments and differ only in their counters ($t+k$ versus k). If $\epsilon = 0$, the shared-keep arm is therefore fresh-state Adam with the multiplier $\rho_k = 1/\kappa(k, t+k)$. This multiplier peaks at 6.57 ($k = 12$) and is 2.35 at $k = 200$. The same holds for `reset_v`, with the whole-update multiplier $\sqrt{b_{t+k}/b_k}$. These identities describe the algorithms. Whether the empirical gain of an artifact arm is explained by its schedule would require a learning-rate-matched run, which we did not perform. (iii) *What remains*. Under $\epsilon = 0$, counter policies differ from one another only by learning-rate schedules. Anything beyond scale must come from changing moment values, as in `reset_m`, `reset_v` and `reset_both`. This is what the update-norm controls test.

Exact correction does not bound the update. After a decoupled `reset_v`, the first step has $\hat{v}_i = g_i^2$ exactly, while the kept first moment gives $\hat{m}_i = (\beta_1 m_i^{\text{old}} + (1 - \beta_1)g_i)/a_{t+1}$. Hence

$$|u_i| = \alpha \frac{|\hat{m}_i|}{|g_i| + \epsilon}, \quad (3)$$

which is unbounded as $|g_i|/|\hat{m}_i| \rightarrow 0$ and is capped only by $\alpha|\hat{m}_i|/\epsilon$. The correction is exact, since $\hat{v}$ is unbiased for a stationary g^2 , but $\hat{v}$ now rests on a single sample. The familiar bound $|u_i| \lesssim \alpha(1 - \beta_1)/\sqrt{1 - \beta_2}$ (Kingma & Ba, 2015) presumes that both moments start from zero together, and it no longer applies. Accuracy of the bias correction and stability of the update are separate properties. For example, coordinates of a unit whose current gradient is (near) zero but whose old momentum is not would receive very large steps. Our logs store only per-tensor norms, so we cannot attribute the observed blow-up to particular coordinates (CHECKPOINT_TRACE_UNAVAILABLE; Section 5.2).

Table 1: Probe AUC (mean held-out probe accuracy over 200 steps) from three starting points, base Adam. Differences are late minus early, per seed paired; brackets give the seed range.

Condition	fresh	early	late	late-early (AUC)	late-early (final acc.)
random-teacher ($C=4$)	0.788	0.727	0.483	-0.245 [-0.315, -0.198]	-0.177 [-0.258, -0.133]
label-permutation ($C=10$)	0.568	0.536	0.332	-0.204 [-0.282, -0.151]	-0.202 [-0.246, -0.137]

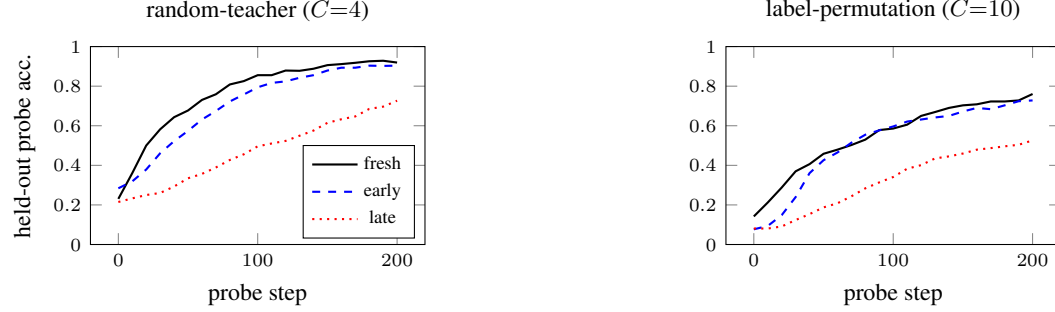


Figure 1: Learning curves on the same never-trained probe task from the fresh, early (after task 0) and late (after task 49) starting points, with identical batches, dropout masks and update budget (mean of three seeds; per-seed curves are in `paper/figures/*.dat`).

5 EXPERIMENTS AND OBSERVATIONS

All numbers are means over the three test seeds, with the per-seed range in brackets. They are descriptive; with three seeds we report neither significance tests nor equivalence.

5.1 A LOSS OF LEARNING ABILITY ON A COMPARABLE NEW TASK

Table 1 and Figure 1 show that the late network learns the probe task more slowly and to a lower final accuracy than the early network, in both conditions and on every seed. The late-minus-early AUC is -0.245 [-0.315, -0.198] for random-teacher and -0.204 [-0.282, -0.151] for label-permutation. The first-50-step accuracy differs by -0.177 and -0.126. By the pre-declared rule the phenomenon is established in both conditions. The early network is itself slightly worse than fresh (-0.060 and -0.032). The fresh comparison reflects the effect of past training on the weights together with a fresh optimizer. It is a diagnostic reference, not a state-only comparison. The gap also appears with the separately tuned $\beta_1=\beta_2=0.9$ Adam (-0.284, -0.283) and SGD with momentum (-0.354, -0.406). In this setting it is therefore not specific to Adam’s default β values.

5.2 STATE-ONLY INTERVENTIONS FROM THE LATE CHECKPOINT

Table 2 summarises the intervention branches. `keep_all` reproduces the late probe of Section 5.1 bit for bit, and all branches received identical batches and masks.

A fresh optimizer state does not recover the gap. `reset_both` with decoupled counters is the same algorithm as re-creating `torch.optim.Adam` (Appendix A). Its difference to `keep_all` is -0.008 and -0.043, far smaller in magnitude than the late-minus-early gap. In this setting, therefore, the loss of learning ability is not explained by stale optimizer state alone.

First-moment reset: no support for benefit beyond update scale. `reset_m` is the only correctly corrected intervention with a positive difference to `keep_all` on every seed: +0.021 [+0.015, +0.032] (random-teacher, which meets the effect criterion) and +0.020 [+0.012, +0.027] (label-permutation, which does not). It also roughly doubles the early update norm (0.0726 versus 0.0311). Against its update-norm control the difference shrinks to +0.012 [+0.010, +0.015] and +0.008, below the minimum interesting effect. We read this as *no support for an additional benefit of at least 0.02* beyond matching the update scale. The random-teacher residual is positive on all three seeds, so we do not claim that it is zero or that the two arms are equivalent.

Table 2: Interventions on the late checkpoint. Δ : probe-AUC difference to `keep_all` and to the intervention’s update-norm control, mean [seed range]. Update norms and descriptive statistics are in Appendix E. [†]Mis-corrected artifact controls (counter not reset), not candidate methods.

Arm	random-teacher ($C=4$)		label-permutation ($C=10$)	
	Δ vs. keep	Δ vs. norm ctrl.	Δ vs. keep	Δ vs. norm ctrl.
<code>keep_all</code>	—	n/a	—	n/a
<code>reset_m</code>	+0.021 [+0.015, +0.032]	+0.012 [+0.010, +0.015]	+0.020 [+0.012, +0.027]	+0.008 [-0.005, +0.021]
<code>reset_v</code>	-0.152 [-0.188, -0.087]	-0.020 [-0.060, +0.056]	-0.244 [-0.290, -0.176]	-0.048 [-0.081, +0.013]
<code>reset_both</code>	-0.008 [-0.049, +0.045]	-0.027 [-0.058, -0.011]	-0.043 [-0.063, -0.008]	-0.030 [-0.050, -0.016]
<code>reset_t</code>	-0.207 [-0.255, -0.183]	+0.000 [+0.000, +0.000]	-0.203 [-0.244, -0.130]	+0.000 [+0.000, +0.000]
<code>reset_v, shared-keep[†]</code>	+0.063 [-0.163, +0.313]	n/a	-0.241 [-0.287, -0.171]	n/a
<code>reset_both, shared-keep[†]</code>	+0.333 [+0.268, +0.410]	n/a	+0.251 [+0.235, +0.273]	n/a

Counter-only reset slows learning, as its schedule predicts. `reset_t` lowers AUC by -0.207 and -0.203, and its early update norm is 0.00572 versus 0.0311 (random-teacher). Consistent with Section 4(i), its accuracy curve ties its update-norm control exactly on every seed. The continuous quantities differ only at the level of $6.5\text{e-}08$ in loss, so the trajectories are close but not bitwise identical (Appendix C). For this arm the control is effectively the same algorithm and does not constitute an independent test.

Exactly corrected second-moment reset is unstable. `reset_v` raises the mean first-10-step update norm from 0.0311 to 16.4 (random-teacher) and from 0.0347 to 330 (label-permutation), and it lowers AUC by -0.152 and -0.244. This is the behaviour equation 3 allows. At the tensor level, the exploding tensors have shared-keep/decoupled update ratios of at most 1.54 (17 of 20 within 1% of 1), instead of the $\epsilon = 0$ prediction 31.62 (Appendix C). By equation 2, this is the signature of ϵ -dominated coordinates, which is consistent with near-zero current gradients meeting a non-zero kept momentum. We did not store per-coordinate traces, so we could not verify this at the coordinate level.

The largest gains come from a mis-corrected artifact. `reset_both` with the counter left unchanged improves AUC by +0.333 and +0.251. This exceeds the late-minus-early gap in random-teacher. Its early update norm is 0.249 versus 0.0311. By Section 4(ii), under $\epsilon = 0$ this arm is fresh-state Adam with a deterministic learning-rate multiplier. We did not run a learning-rate-matched control, so we do not attribute its empirical gain entirely to the learning rate. The observation does show that, in this setting, the late network’s probe learning is highly sensitive to update scale.

Descriptive statistics. All branches update their weights after the intervention. Relative parameter movement, CKA of hidden features to the starting point, and the accuracy drop on task 49 (Appendix E) vary with update scale. For example, the artifact arm moves the parameters most and has the lowest CKA. We use these only as descriptions, not as evidence that features were or were not relearned.

6 DISCUSSION AND LIMITATIONS

What the observations support. In our setting there is a clear loss of learning ability on a comparable new task. A state-only intervention with exact bias correction did not produce a benefit that survives matching its update scale. The counter-only reset reduces, in the $\epsilon = 0$ idealisation, to a learning-rate schedule. The largest improvements came from arms that take much larger steps. Taken together, these point toward update scale, not stale optimizer statistics, as the quantity to control first when asking whether state repair helps. We regard this as a hypothesis for further testing, not as a conclusion.

What they do not support. We do not claim that optimizer-state repair cannot restore plasticity, that the `reset_m` residual is zero, or that the artifact arm’s gain is entirely due to its learning rate. Non-significance at three seeds is not equivalence. We also make no claim about frozen or preserved features: every branch updates its weights, and CKA and parameter distance are descriptive only.

Limitations.

- **Oracle timing.** Every intervention is applied at a pre-specified task boundary. Nothing here evaluates a boundary-free or online trigger.
- **Objective mismatch.** Learning rates were tuned for online accuracy over the whole stream, while the probe metric is the learning speed on one new task from a late checkpoint. A learning rate tuned for the probe could shrink or remove the gap.
- **One probe task per condition and two synthetic conditions.** Tasks, checkpoints and arms are not independent replicates; the unit of replication is the seed.
- **Three previously inspected seeds.** Seeds 3000–3002 were used in an earlier smoke run, so this study is exploratory.
- **Small scale, no real data.** A 32-unit MLP trained in pure Python. We used no image or text benchmark and no pretrained weights.
- **Controls not run.** There is no learning-rate-matched control for the artifact arms, no probe-specific learning-rate tuning, and no coordinate-level traces (Appendix F).
- ϵ . The scalar equivalences of Section 4 hold only with $\epsilon = 0$. Their accuracy at $\epsilon = 10^{-8}$ is checked only at the level of per-tensor norms in our logs.

7 CONCLUSION

We set out to separate optimizer-state interventions from their effect on update scale in a setting where the loss of learning ability is actually observed. The loss appears on both synthetic streams. Replacing the optimizer state with a fresh one does not recover it. The only correctly corrected intervention with a consistent positive difference does not exceed its update-norm control by our minimum interesting effect. The largest gains come from mis-corrected arms that take much larger steps. On this evidence we pause work on optimizer-state repair as a method in this setting. The draft is offered as a careful diagnostic of one small setting, with its algebra and controls made explicit, and not as evidence about optimizer-state repair in general.

AI USE STATEMENT

Generative AI (Claude Code, an AI coding agent) was used for most of this project:

- writing all experiment code and tests;
- running the experiments;
- checking prior work through web retrieval, with per-source verification levels recorded in the repository file `PRIOR_ART.md`;
- deriving the algebra of Section 4;
- drafting this entire manuscript.

The AI agent verified the code with unit tests and with a numerical comparison against PyTorch. Every number in the text is generated from raw result files by a script. **No human author has yet reviewed the code, the analysis, the citations or this text (HUMAN_REVIEW_PENDING).** In particular, the citations have not been independently re-checked against the published versions. This statement will be revised once human review has taken place; the authors will then take responsibility for the final content.

REPRODUCIBILITY STATEMENT

The configuration was fixed and committed before the run (`configs/p4_stage1.json`, sha256 prefix `d9c2b7aceeeb`; code `0404c14`, code hash `b45b507e4aef`). The run manifest records environment, CPU time, peak memory, per-cell status and data and checkpoint hashes. Raw logs contain per-step training records and per-evaluation probe records. Experiments use only the Python standard library and a single CPU thread. PyTorch is used only for parity tests. The exact commands are in `paper/BUILD.md` and `STATUS.md`.

REFERENCES

- Kavosh Asadi, Rasool Fakoor, and Shoham Sabach. Resetting the optimizer in deep RL: An empirical study. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.17833.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mah-mood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024. doi: 10.1038/s41586-024-07711-7.
- B. Ellis, M. T. Jackson, A. Lupu, A. D. Goldie, M. Fellows, S. Whiteson, and J. N. Foerster. Adam on local time: Addressing nonstationarity in RL with relative Adam timesteps. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2412.17113.
- A. Galashov, M. K. Titsias, A. György, C. Lyle, R. Pascanu, Y. W. Teh, and M. Sahani. Non-stationary learning of neural networks with automatic soft parameter reset. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2411.04034.
- J. F. Hernandez-Garcia, T. Figliolia, and B. Millidge. Can scale save us from plasticity loss in large language models?, 2026. arXiv:2606.24752.
- Liwei Kang and Wee Sun Lee. Continual learning of numerous tasks from long-tail distributions, 2024. arXiv:2404.02754.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980.
- T. Klein, C. Luther, M. McAuliffe, L. Miklautz, C. Plant, and S. Tschitschek. Plasticity loss in deep reinforcement learning: A survey, 2024. arXiv:2411.04832 (v3, 2026; author list taken from v3).
- Saurabh Kumar, Henrik Marklund, and Benjamin Van Roy. Maintaining plasticity in continual learning via regenerative regularization. In *Conference on Lifelong Learning Agents*, 2024. arXiv:2308.11958.
- H. Lee, H. Cho, H. Kim, D. Kim, D. Min, J. Choo, and C. Lyle. Slow and steady wins the race: Maintaining plasticity with hare and tortoise networks. In *International Conference on Machine Learning*, 2024. arXiv:2406.02596.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *International Conference on Machine Learning*, 2023. arXiv:2303.01486.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, Hado van Hasselt, Razvan Pascanu, James Martens, and Will Dabney. Disentangling the causes of plasticity loss in neural networks. In *Conference on Lifelong Learning Agents*, volume 274 of *Proceedings of Machine Learning Research*, pp. 750–783, 2024. arXiv:2402.18762.
- S. Sahu, A. Sarkar, C. J. Hogan, and M. T. Wells. Adapt or forget: Provable tradeoffs between Adam and SGD in nonstationary optimization, 2026. arXiv:2605.04269.
- Jiuqi Wang, Jayanth Srinivasa, Claire Chen, Shuze Daniel Liu, Ali Payani, and Shangdong Zhang. Predicting plasticity in deep continual learning: A theoretical perspective, 2026. arXiv:2605.09044.

A IMPLEMENTATION CHECKS

The pure-Python Adam was compared with `torch.optim.Adam` (PyTorch 2.14.0, CPU, float64, `foreach`, fused, `capturable` and `AMSGrad` off) on a 400-step gradient sequence. The maximum relative differences were $1.7\text{e-}14$ (weights), $1.4\text{e-}13$ (m), $3.9\text{e-}16$ (v) and $3.6\text{e-}11$ (updates), and the step counters were identical. Trace comparisons further show that the following hold within numerical tolerance:

- `reset_both` with decoupled counters equals a newly constructed torch optimizer;
- `reset_t` equals zeroing torch’s step;
- the shared-keep variants equal zeroing `exp_avg/exp_avg_sq` alone.

Decoupled single-moment resets have no stock-PyTorch equivalent and are checked only against their explicit formulas. These are implementation checks, not proofs.

B COUNTEREXAMPLE TO SCALAR COUNTER EQUIVALENCE WITH $\epsilon > 0$

Two coordinates receive the constant gradients 1 and 10^{-8} for 10,000 steps ($\beta_1=0.9$, $\beta_2=0.999$). From the same post-gradient (m, v) , one further step is taken with the counter kept ($t=10,001$) and with the counter reset ($r=1$). With $\epsilon = 0$ the ratio of the reset update to the kept update is 0.316235 for the first coordinate and 0.316235 for the second. This equals $\kappa(1, 10,001) = 0.316235$, which differs from the $t \rightarrow \infty$ value 0.316228 by the finite-age factor. With $\epsilon = 10^{-8}$ the ratios are 0.316235 and 0.613082. The two coordinates are therefore rescaled differently, and no single learning-rate multiplier reproduces the counter reset. The implementation matches equation 2 to a relative error of $2.2\text{e-}16$. The same behaviour is observed with stock `torch.optim.Adam` when `step` is zeroed (`tests/test_torch_parity.py`). This is a sanity check of elementary algebra, not a contribution. Script: `analysis/counter_equivalence.py`; output: `runs/p4_analysis_counter/result.json`.

C RAW-TRACE PRECISION OF THE `RESET_T` CONTROL

Only the first step after the intervention shares (m, v) across branches, so one-step ratios are checked at that step only, per tensor, over both conditions and three seeds.

- `reset_t/keep_all`: the maximum relative deviation from $\kappa(1, t+1)$ is $8.1\text{e-}07$.
- `reset_both` shared-keep/decoupled: the ratio is predicted to be 3.162 at $\epsilon = 0$; the maximum relative deviation is $4.6\text{e-}04$.
- `reset_v` shared-keep/decoupled: the ratio is predicted to be 31.62 at $\epsilon = 0$, but the observed ratios range from 1 to 31.6.

For `reset_v`, 17 of 30 tensor-seed pairs have a ratio within 1% of 1. Of the 20 pairs whose decoupled update norm exceeds 1, 17 are within 1% of 1 and none exceeds 1.54, far below the $\epsilon = 0$ prediction. By equation 2, a ratio of 1 is what results when ϵ dominates $\sqrt{\hat{v}}$ in both branches. The output bias, whose gradient is never structurally zero, has the predicted ratio (at least 31.62). This is per-tensor evidence consistent with near-zero-gradient coordinates driving the `reset_v` blow-up. It is not a coordinate-level verification (`CHECKPOINT_TRACE_UNAVAILABLE`).

Over the whole 200-step trajectories, `reset_t` and its update-norm control differ by at most $6.5\text{e-}08$ in training loss, $5.4\text{e-}08$ in probe loss and $9.8\text{e-}09$ in relative parameter change, and by $0.0\text{e+}00$ in probe accuracy. The “+0.000” in Table 2 is thus an exact tie in accuracy between trajectories that are not bitwise identical. The small continuous differences are consistent with $\epsilon > 0$ and rounding. The control computes its own update direction and rescales only its per-tensor norm (`osrepair/runner.py, make_norm_matcher`); it never copies the reference direction.

D TUNING

Online training accuracy on the development seeds (1000, 1001) for each learning rate. All selections were interior grid points, so no edge extension was used.

Condition	Family	selected LR	dev online acc. by LR
RT	adam	0.003	0.001: 0.451, 0.003: 0.544, 0.01: 0.519, 0.03: 0.400
RT	adam_eqbeta	0.003	0.001: 0.414, 0.003: 0.478, 0.01: 0.452, 0.03: 0.384
RT	sgd	0.01	0.003: 0.492, 0.01: 0.513, 0.03: 0.451, 0.1: 0.320
LP	adam	0.003	0.001: 0.273, 0.003: 0.361, 0.01: 0.347, 0.03: 0.237
LP	adam_eqbeta	0.003	0.001: 0.228, 0.003: 0.269, 0.01: 0.241, 0.03: 0.192
LP	sgd	0.01	0.003: 0.303, 0.01: 0.347, 0.03: 0.309, 0.1: 0.189

E DESCRIPTIVE STATISTICS OF THE INTERVENTION BRANCHES

Mean update norm over the first 10 probe steps ($\|\bar{u}\|_{1:10}$), relative parameter change after 200 probe steps, linear CKA of hidden features to the starting point on 64 fixed inputs, and accuracy drop on task 49 (mean of three seeds). These are descriptive only.

Arm	random-teacher ($C=4$)				label-permutation ($C=10$)			
	$\ \bar{u}\ _{1:10}$	rel. $\Delta\theta$	CKA	old drop	$\ \bar{u}\ _{1:10}$	rel. $\Delta\theta$	CKA	old drop
keep_all	0.0311	0.38	0.703	0.441	0.0347	0.342	0.773	0.398
reset_m	0.0726	0.391	0.689	0.449	0.0808	0.349	0.786	0.408
reset_v	16.4	15.1	0.351	0.418	330	274	0.159	0.365
reset_both	0.049	0.448	0.756	0.444	0.0403	0.386	0.845	0.400
reset_t	0.00572	0.169	0.925	0.143	0.00632	0.16	0.903	0.109
reset_v, shared-keep [†]	17.3	15.7	0.294	0.487	331	273	0.160	0.368
reset_both, shared-keep [†]	0.249	0.859	0.471	0.538	0.209	0.711	0.723	0.426

F PLANNED EXPERIMENTS (NOT RUN)

None of the following has been run, and no result is implied:

1. a learning-rate-matched control for the shared-keep artifact arms, using the schedule of Section 4(ii);
2. tuning the learning rate for the probe objective, to test whether the late-minus-early gap persists;
3. more probe tasks per condition, fresh test seeds that have not been inspected, and a real-data stream;
4. coordinate-level traces at the late checkpoint, to test the near-zero-gradient explanation of `reset_v`;
5. any boundary-free trigger or predictor of intervention benefit (deliberately out of scope).

G CORRECTION HISTORY

An earlier internal report (`docs/STAGE1_REPORT.md`) stated that `reset_t` equals a per-tensor learning-rate rescaling “exactly” and that the first-step ratio is 0.3162. Both statements are corrected here. The equivalence holds only for $\epsilon = 0$ and coordinates with $v_i > 0$, and 0.3162 is the $r = 1$, $t \rightarrow \infty$ approximation of $\kappa(r, t)$ in equation 2. The earlier report also did not distinguish one-step equivalence from trajectory equivalence. The observed “0” is an exact tie in accuracy while continuous quantities differ at about 6.5e-08 (Appendix C). The original report is kept unchanged in the repository, with a pointer to this correction.